

# Continuing previous project

Source: <https://claude.ai/chat/8df8d653-4693-4669-95f3-7f87fc8e45ed>

Captured: 9/5/2026, 8:33:22 PM

Complete indexed sequence captured (76 message rows).

---

**YOU**

Can we continue this project, starting where we left off in the last chat.

## CLAUDE

I've read the three files and picked up the thread from the last two chats. Here's where things actually stand, plus one gap I found.

### State

`main.py` (420 lines) and `index.html` (1,228 lines) on disk match what the record describes: eager attention, `@torch.no_grad()` on all four model-touching functions, `/api/tokenize`, the dual-lens table, the attention grids. The record's §7.7 has real measured attention numbers, so the last unverified piece from the previous chat — the attention test — did complete. **No code is blocking.** [retrieved: uploaded files + record §13]

Open work per record §13: attention-exploration guidance panel, a deliberate failure case, rehearsing the §9 talk track, and optionally a position selector for the lens table.

### The gap

The two panel rewrites decided in the *other* chat were never applied to `index.html`. The file still has the pre-correction text.

Current, verbatim, lines 604–611:

The leading explanation is that softmax forces every row to sum to 1, so a head with nothing particular to attend to has to put its budget somewhere, and the first position serves as a default. Treat a bright first column as probably *not* telling you something about that word's meaning.

Two problems, both identified in that chat and both still live in the file:

1. "The leading explanation" asserts a consensus that doesn't exist — there are at least three live accounts.
2. It frames the sink as noise to discount. That's backwards for your demo. A pattern nobody designed, that every model develops, that breaks the model when removed, is *harder* for a skeptic to call random than something you tell them to ignore.

The replacement paragraph was written out in full in that chat, along with a full rewrite of the "Why 9 heads" panel (which currently says heads run the computation "9 times in parallel" — the rewrite corrects this to heads as 64-dimensional *slices*,  $576 = 9 \times 64$ ). Both are retrieved text, not something I'd be regenerating.

Record §10 says the sink is "stated in the UI as a leading explanation, not a settled one." The file says "The leading explanation is." Those aren't the same claim — the record is describing the intended text, not the shipped text.

## What I'd do next

Apply the two panel rewrites first. They're already-written text, zero risk, and they fix a claim your coworkers could reasonably challenge.

Then the two attention display changes from that chat's ranked list, because §13 item 1 is explicitly blocked on "someone has actually explored the grids" and right now the grids are hard to read:

- **Uniform-baseline overlay** — show each column-o weight as a multiple of  $1/(i+1)$  instead of raw. Turns "that column looks bright" into a number in the same idiom as your  $17,803\times$  line.
- **Max-across-heads** as a third aggregation option alongside mean. The mean smooths away exactly the specialization you want to find. ~4 lines.

With those two in, the middle-layer / individual-head exploration that §7.7 flags as unverified becomes tractable, and item 1 unblocks itself.

Say the word and I'll produce the panel edits as targeted `str_replace` -style diffs rather than a full-file rewrite — the previous chat noted full rewrites were the thing consuming context.

---

**Solid** — Panel text quoted above is verbatim from the uploaded `index.html` (lines 604–611, 581–599); `main.py` state confirmed by reading it here; §13/§10 items read directly from the record; replacement panel text retrieved from the prior conversation, not reconstructed.

**Uncertain / contested** — Whether you've made edits to either file since the 15:18 upload; if so, my line numbers are stale. Also: I have *not* re-verified the Llama-2-13B perplexity figure (5.40 → 5158.07) cited in the prior chat as evidence sinks are load-bearing — that was a web-search result in that conversation, and it's the single input most affecting whether the "not decoration" framing in the replacement text holds up under challenge.

**To verify independently** — (1) Confirm your local `index.html` still contains the string "The leading explanation is that softmax"; (2) re-check the StreamingLLM perplexity number before you say it out loud to coworkers; (3) run the attention view at layers 12–20 on individual heads before I write the guidance panel, so it describes what's there.

**YOU**

Can you double check to make sure the index.html file wasn't updated?

Does this look like the correct code(maybe I forgot to save before running the last test?)?

CLAUDE

Checked. The file you pasted is the same file that's on disk here — and the panel text was never changed in the first place.

Fingerprint comparison

| Check                                                                  | Uploaded file                    | Your pasted text |
|------------------------------------------------------------------------|----------------------------------|------------------|
| Size / lines                                                           | 52,605 bytes, 1,228 lines        | —                |
| MD5                                                                    | ae3c51559b7725347c3b975b6504e443 | —                |
| "The leading explanation is that softmax forces every row to sum to 1" | present                          | present          |
| "Each layer runs attention <b>9 times in parallel</b> "                | present                          | present          |
| }function renderAttention() { (line 1143, missing line break)          | present                          | present          |

That last one is the useful signal. It's a cosmetic quirk — the closing brace of `loadAttention` and the opening of `renderAttention` are jammed onto one line — and it appears identically in both. A quirk that specific surviving a copy is strong evidence you pasted the same bytes. [computed: greps against the uploaded file; visual match against your paste]

I also grepped for strings that would only exist if the rewrites had landed: `1/(i+1)`, "max across", "value norm", "over-mixing", "StreamingLLM", "perplexity", "576 = 9", "plumbing" — **zero hits each**. The five hits for "uniform" are all the pre-existing `UNIFORM = 1 / VOCAB` sampling baseline, and the one "slice" hit is `.slice(0, 3)` in the attention bottom-row code. No trace of the rewrites anywhere. [computed]

Correcting the premise

maybe I forgot to save before running the last test?

That isn't what happened, and it's worth being clear about because the alternative explanation would imply lost work.

The rewrites were never applied to any file. They were written out as *suggested* replacement text in the **other** chat — "Understanding transformers and Llama architecture" — during a conceptual discussion of attention sinks. That thread never touched `index.html`. The demo-build thread, where the file edits actually happened, ended before that discussion and never saw the corrected text.

Two threads, one file, and the correction landed in the thread that wasn't editing it.

Separately: the attention test *did* run successfully against this exact file. §7.7 of the record has real grids and a bottom-row summary in precisely the format line 1189–1194 produces. So there's no unsaved-file scenario there either — the code you have is the code that produced those numbers.

Nothing is lost. The replacement text is retrievable from the other conversation, which I did last turn.

---

**Solid** — Uploaded `index.html` is 52,605 bytes / 1,228 lines, MD5

`ae3c51559b7725347c3b975b6504e443` ; contains both pre-correction panels; contains the `}function renderAttention` formatting quirk that also appears in your paste; contains none of eight distinctive strings that the rewrites would have introduced.

**Uncertain / contested** — I did **not** byte-compare your pasted text against the file. I wrote your paste to disk nowhere and hashed nothing of it; retyping it myself would have introduced my own errors. What I did was compare fingerprints, including one arbitrary formatting quirk. That's strong but not proof of byte-identity. The single input most affecting this conclusion is whether the copy you pasted came from `C:\Projects\llm-demo\static\index.html` or from somewhere else.

**To verify independently** — In `cmd.exe`: `certutil -hashfile C:\Projects\llm-demo\static\index.html MD5` and compare against `ae3c51559b7725347c3b975b6504e443`. If it matches, the question is fully closed. If it doesn't, your local file has changed since the 15:18 upload and I'm working from a stale copy — tell me and I'll re-fingerprint.

**YOU**

I believe we're good to continue. I was mistaken

**CLAUDE**

Verified the empirical claim first, since it's the load-bearing one in the new panel.

The StreamingLLM paper measured perplexity on Llama-2-13B over the first book (65K tokens) of the PG-19 test set: a cache holding 1,024 recent tokens and no initial ones scored 5158.07, while keeping four initial tokens alongside 1,020 recent ones scored 5.40, and substituting those four initial tokens with line-break tokens scored 5.60. The paper's own framing is that the sink emerges because initial tokens draw strong attention even when they carry no semantic importance. [arXiv](#)

That last number — line breaks working nearly as well as real words — is the better demo point. It's direct evidence the sink is about *position*, not meaning, and you can say it without hand-waving.

One correction to how I described this last turn: I said "removing sinks at inference time degrades model performance." More precisely, this is a *sliding-window cache* experiment. The initial tokens get evicted as the window slides; it isn't a surgical removal from a full-context model. The conclusion holds, but the setup matters if anyone asks.

Now applying both rewrites.

**YOU**

CLAUDE

That's a clean reproduction of §7.7 — same six numbers in the column, same bottom-row summary (65.4 / 14.3 / 11.3). Same prompt, same weights, same output. Worth keeping as a determinism beat for the talk.

One thing the image can't tell me: the panels are collapsed, so I can't confirm from this whether the new text landed. Expanding "Two things you'll see" should now start with "Some of that brightness is forced by arithmetic."

What the grid actually says

Read row, column 0 looks like it's *fading* down the prompt: 100 → 79 → 75 → 66 → 62 → 65. Against the uniform baseline it's doing the opposite.

| row     | i | shown | uniform 1/(i+1) | ratio |
|---------|---|-------|-----------------|-------|
| The     | 0 | 100%  | 100.0%          | 1.00× |
| capital | 1 | 79%   | 50.0%           | 1.58× |
| of      | 2 | 75%   | 33.3%           | 2.25× |
| Japan   | 3 | 66%   | 25.0%           | 2.64× |
| is      | 4 | 62%   | 20.0%           | 3.10× |
| called  | 5 | 65%   | 16.7%           | 3.90× |

The sink *strengthens* monotonically as the row gets more alternatives to choose from. The raw numbers hide that completely, and the colour scale hides it worse — 79 and 65 render at nearly the same brightness while sitting at 1.6× and 3.9×.

Two consequences:

**Row 0 is not evidence of anything.** It's 100% because it's the only unmasked cell in the row. If a coworker points at the top-left square as the sink, that's the one cell where the correct answer is "arithmetic, not behaviour." The sink case rests on rows 1–5.

**The Japan→France test can't work in this view.** No row attends to the Japan column above 10% except Japan itself. Column 3 is empty from row 4 down. Swapping in France would change nothing visible here — which is the null result §7.7 anticipated, now confirmed on your

screen rather than predicted. It doesn't rule out a head somewhere doing it; the mean of 9 is exactly the aggregation that would erase a single head's behaviour.

Bottom row otherwise: 14% on itself, 11% on `is` (the previous token), ~10% spread across the three cells too small to print.

## The case for building the overlay now

This screenshot is the argument for it. You have a grid where the eye reads the trend backwards, and the fix is one line of arithmetic you already trust. Displaying each column-0 cell as a multiple of  $1/(i+1)$  turns "that column looks bright" into  $3.90\times$  — the same move as your  $17,803\times$  line, which is the number that lands hardest in the rest of the demo.

Paired with max-across-heads, you'd also be able to ask the Japan question properly: if any of the 9 heads at any layer attends to position 3 from position 5, max-across surfaces it and mean buries it.

Want me to write both into `renderAttention()` ? It's a scoped edit to that one function — the layer/head plumbing and the fetch path don't change.

---

**Solid** — All six ratios recomputed from the screenshot values against  $1/(i+1)$ ; the six column-0 percentages and the bottom-row triple match record \$7.7 exactly; column 3 has no printed value below row 3, so no row attends to `Japan` above the 10% print threshold.

**Uncertain / contested** — Displayed cells are rounded to whole percent and suppressed below 10%, so the ratios above carry roughly  $\pm 0.5$  pp of display error and the three blank cells in row 5 are only known to sum to ~10 pp. Hovering gives the exact figures. Also: this is one layer, one aggregation, one prompt — the monotonic strengthening is real in this grid, but whether it holds across depth is untested and is the single thing most affecting whether the overlay tells a clean story.

**To verify independently** — (1) Expand the two attention panels and confirm the new text is there; (2) hover row 5 / column 0 for the exact weight and check it against 65.4%; (3) step through layers 1, 6, 12, 18, 24 at mean-of-9 and see whether the ratio trend survives, before I build a display that assumes it does.

**YOU**

Here's the text included: 6. Attention — which tokens looked at which How to read this grid This is the one view in the whole tool with genuinely meaningful axes. Both are token positions from your prompt. Rows are the token doing the looking. Columns are the token being looked at. Cell (row  $i$ , column  $j$ ) is how much of position  $i$ 's attention went to position  $j$ . Brighter means more. Every row sums to 1. Each position distributes a fixed budget of attention across the positions available to it. The grey upper triangle is the causal mask. Position 2 cannot look at position 5, because position 5 hasn't happened yet. That's not a display choice — those values are mathematically zero. It's the same constraint that lets generation cache earlier positions instead of recomputing them. The bottom row is usually the interesting one: it's the position that produces the next-token prediction. Why 9 heads, and what "grouped-query attention" means here A head is a slice, not a copy.  $576 = 9 \times 64$ : each of the 9 heads works in its own 64-dimensional subspace. The layer isn't doing the full computation nine times — it's doing nine narrow computations at once. Why nine? One head produces exactly one set of attention weights per position, so it can look back at the context in exactly one way. Nine heads let the layer read the context nine ways simultaneously. The cost is that each read gets 64 dimensions instead of 576. Each head's output goes through its own slice of the output matrix and is added to the residual stream. So a layer's attention step is really 9 independent writes summed together — the same additive structure as the residual stream itself, one level down. Grouped-query attention: 9 query heads, 3 key/value heads, group size 3. The grids below are per query head, so there are 9 of them. Within a group, the three heads read from the same keys and values but write through different output slices — so GQA constrains what a group of heads can read, not what they can write. At the prompt lengths in this demo, the restriction on achievable attention patterns is essentially nil. The point of it is the KV cache during generation:  $3 \times 64 \times 2 \times 4 \text{ bytes} \times 30 \text{ layers} = 45 \text{ KiB}$  per token here, versus 135 KiB if every head had its own keys and values. That matters enormously for a large model serving long contexts, and matters not at all for a six-token prompt on a laptop. You're looking at a design decision made for a situation you aren't in. Do heads specialize? In the models researchers have studied closely, yes — previous-token heads, induction heads and positional heads are all documented. Whether any specific head in this model has a clean job is unknown; nobody has published a map of these 270 heads. "Mean of all heads" averages the 9 grids. Useful for a quick overview, but it smooths away exactly the specialization that makes individual heads interesting. Two things you'll see that are easy to misread Load attention Layer 30 Head mean of all 9 Try: run this with "Japan" in your prompt, note where the last row is looking, then change Japan to France and reload. Whether the pattern visibly reroutes is something to check rather than assume — small models are noisier than the textbook pictures. 30 layers  $\times$  9 query heads = 270 grids, all loaded. Switching between them costs no server request. The capital of Japan is called The 100 capital 79 21 of 75 20 Japan 66 21 is

62 25 called 65 11 14 Rows sum to 1. Grey cells are causally masked — those values are exactly zero. Numbers shown only where attention exceeds 10%. Hover any cell for the exact figure. Bottom row (the position that predicts the next token) is looking at: The #0 (65.4%), called #5 (14.3%), is #4 (11.3%).

**CLAUDE**

The 9-heads panel is showing the new text — "A head is a slice, not a copy," the o\_proj additive point, the 45 KiB line. So the patch landed.

That also settles the sink panel without needing to see it. Both replacements happened in a single atomic write: the script checked six post-conditions and would have exited without writing if either one failed. The heads panel being new means the sink panel is new. Its body is missing from your paste because `<details>` content that's collapsed doesn't get picked up by a copy selection — you evidently had the first two expanded and the third closed.

Still worth expanding once and eyeballing it, since that's the paragraph a coworker will push on.

## One inconsistency the screenshot exposed

The hint under the controls is now pointing people at a test that can't work:

Try: run this with "Japan" in your prompt, note where the last row is looking, then change Japan to France and reload.

At layer 30 / mean-of-9 — which is what the page loads by default — column 3 is empty from row 4 down. Nothing attends to Japan. A coworker who follows that instruction literally will swap Japan for France, see nothing change, and reasonably conclude the tool doesn't work or that attention doesn't mean anything.

The hint predates the measurement. It should either name the layers and heads worth trying, or say plainly that the default view is the wrong place to look. I'd rather fix it after the overlay exists, since the overlay is what makes "the wrong place to look" legible rather than just asserted.

So, in order: overlay + max-across-heads in `renderAttention()`, then rewrite the hint to point at whatever that reveals. Say go and I'll do the first part.

---

**Solid** — New 9-heads text confirmed present in your rendered page; both panel replacements were written in one atomic operation gated on six post-conditions, so the sink panel is necessarily updated too; column 3 receives no printed weight from rows 4 or 5 in the layer-30 mean grid.

**Uncertain / contested** — I have not seen the sink panel body rendered. The atomic-write argument is strong but it's an inference, not an observation. Also unknown: whether some *individual* head reroutes on Japan→France, which is the thing that determines whether the hint gets rewritten or deleted.

**To verify independently** — Expand "Two things you'll see that are easy to misread" and check it opens with "Some of that brightness is forced by arithmetic."

## YOU

I accidentally missed it. Here it is:

Two things you'll see that are easy to misread The first column is often very bright, in many heads at once. Some of that brightness is forced by arithmetic. Row  $i$  has only  $i+1$  unmasked cells, so even perfectly uniform attention would put  $1/(i+1)$  on column 0 — 100% for row 0, 50% for row 1, down to about 17% for row 5 on a six-token prompt. Compare what you see against that baseline, not against zero. The excess is the attention sink: a documented pattern where the first position absorbs large amounts of attention across many heads regardless of what that token actually is. The mechanism is usually attributed to softmax — every row must sum to 1, so a head with nothing in particular to attend to still has to put its budget somewhere, and position 0 is the only position every row can see. Why models learn to do this is more contested; one line of work argues it keeps representations from collapsing into each other over many layers. It is not decoration. In the experiment that named the phenomenon (Xiao et al., StreamingLLM, ICLR 2024), a sliding-window cache holding 1,024 recent tokens but no initial ones sent Llama-2-13B's perplexity to 5158.07 — total collapse. Keeping just four initial tokens alongside 1,020 recent ones restored it to 5.40. Replacing those four with plain line-break tokens restored it almost as well, to 5.60, which tells you the sink is about position, not about what those particular words mean. So a bright first column is telling you about the head's plumbing, not about that word's meaning — but the plumbing is load-bearing, and nobody designed it. The model learned it during training, and every model anyone has looked at does it. The diagonal is often bright too — positions attending to themselves. Also usually structural rather than semantic. The interesting signal is off-diagonal, non-first-column brightness: a later token reaching back to a specific earlier token.

**CLAUDE**

Both panels confirmed live. That closes the loop — the correction from the other chat is finally in the file it was written for.

Two sentences in that text I'd tighten before a skeptic reads them, both mine, both places where I wrote more confidently than my sourcing supports.

**1. The last line of the third paragraph.**

and every model anyone has looked at does it

I flagged this as overstated last turn and it's still there. What I actually retrieved is the StreamingLLM stress test, which covered Llama-2 at 7B/13B/70B, Falcon at 7B/40B, MPT at 7B/30B, and Pythia at 2.8B/6.9B/12B. That's ten models across four families — plenty strong, and checkable. Replace with:

The model learned it during training, and it turns up in every model that paper tested — ten of them, across the Llama-2, Falcon, MPT and Pythia families.

Trades an unfalsifiable universal for a number someone can go verify. Better rhetoric anyway.

**2. The opening of that paragraph.**

In the experiment that named the phenomenon

That's a priority claim I didn't check. The paper certainly made the term standard, but whether it coined it is something I'd be asserting from vibes. Replace with:

In the study that made the term standard

Or drop the framing entirely and let the parenthetical citation carry it.

Neither is load-bearing for the argument. Both are the kind of thing that costs you credibility if someone checks and finds it slightly off, and costs nothing to fix.

I'd fold both into the same edit as the `renderAttention()` work so you drop one file rather than three. Say go and I'll do the overlay, max-across-heads, and these two sentences in one pass.

---

**Solid** — Both replacement panels are rendering as written; the ten-model list and four family names come from the StreamingLLM material retrieved earlier this conversation.

**Uncertain / contested** — Whether Xiao et al. originated the term "attention sink" or adopted it; I have not checked, which is exactly why I'd remove the claim rather than defend it. The over-mixing sentence remains sourced only to a prior conversation.

**To verify independently** — Search the StreamingLLM paper for the first use of "attention sink" and whether it cites an earlier source for the term, if you want to keep a priority claim at all.

**YOU**

Where do I find the StreamingLLM paper?

**CLAUDE**

All four of these came back in the search I ran earlier in this conversation.

**YOU**

Can you please research and verify for me?

## CLAUDE

Fetches the full HTML. All four questions answered, and one of my cautions turns out to have been unnecessary.

### 1. Table number — confirmed

The perplexity figures are in **Table 1**, inside Section 3.1 under the heading "LLMs attend to Initial Tokens as Attention Sinks." The caption states that window attention performs poorly on long text, that perplexity is restored when the initial four tokens are reintroduced alongside the recent 1020 ( $4+1020$ ), and that substituting those four with linebreak tokens ( $4"\n"+1020$ ) achieves comparable restoration; the config notation  $x+y$  means  $x$  initial tokens with  $y$  recent ones, and perplexities are measured on the first book (65K tokens) of the PG19 test set. The Llama-2-13B values are  $0+1024$  (Window) 5158.07,  $4+1020$  5.40, and  $4"\n"+1020$  5.60.

My hedge was unnecessary. And your panel's phrasing — "a sliding-window cache holding 1,024 recent tokens but no initial ones," "four initial tokens alongside 1,020 recent ones" — matches the table's own definitions exactly. No change needed there.

### 2. Term origin — they coined it, so keep the claim

In the Introduction, describing the disproportionate attention allocated to initial tokens irrespective of relevance, the authors write that they term these tokens "attention sinks." Section 3.1 repeats it as introducing the concept. Appendix B goes further, distinguishing their contribution from concurrent work: they note that Han et al. (2023) independently studied length-generalization failure with a similar  $\Delta$ -shaped attention approach, but claim the attention-sink phenomenon they uncover extends beyond length generalization, appearing in encoder transformers like BERT and in Vision Transformers.

So **"the experiment that named the phenomenon" is accurate and should stay**. I was wrong to flag it. Worth being precise about what it means, though: they named it, they didn't first observe the underlying oddity — the paper itself points to prior observations of the same thing in quantization-outlier work and to Miller's SoftMax-Off-by-One as a proposed remedy. "Named" is exactly the right verb. "Discovered" would not be.

### 3. Authors — "Xiao et al." is correct

Guangxuan Xiao (MIT, part of the work during a Meta AI internship), Yuandong Tian (Meta AI), Beidi Chen (CMU), Song Han (MIT / NVIDIA), Mike Lewis (Meta AI).

## 4. Ten models — confirmed, with a caveat

Section 4.1 states StreamingLLM handles texts beyond 4 million tokens across Llama-2-[7,13,70]B, Falcon-[7,40]B, Pythia-[2.8,6.9,12]B, and MPT-[7,30]B. That's  $3 + 2 + 3 + 2 = 10$ .

Caveat: the Introduction writes the Pythia sizes as [2.9, 6.9, 12] while Section 4.1 writes [2.8, 6.9, 12]. 2.8B is the real Pythia size, so the Introduction has a typo. Don't quote 2.9.

## What I'd change in the panel

Only the one sentence. Replace:

The model learned it during training, and every model anyone has looked at does it.

with:

The model learned it during training, and the same paper found it in all ten models it tested — across the Llama-2, Falcon, MPT and Pythia families — and in BERT as well.

Appendix H confirms the BERT finding: BERT-base-uncased shows disproportionately high attention on the [SEP] token in most layers. That's a nice addition, because a decoder-only explanation that also holds for an encoder is harder to dismiss as an artifact of causal masking.

## The one thing still unverified

Your panel's clause "one line of work argues it keeps representations from collapsing into each other over many layers" is **not** from this paper. StreamingLLM's own account is purely the softmax-denominator one plus the visibility argument for why *initial* tokens specifically. The over-mixing claim traces to a different paper and is still sourced only to a prior conversation. Say the word and I'll check that one too.

## A testable prediction for your layer sweep

This is the useful find. Figure 2 reports that in Llama-2-7B the attention maps in layers 0 and 1 show a local pattern with recent tokens receiving more attention, and that beyond those bottom two layers the model heavily attends to the initial token across all layers and heads. Appendix F quantifies it at length 4096: the 4096th token's attention to the first token is significantly high, often exceeding half the total, except in the two bottom layers.

SmolLM2 is a decoder-only Llama-architecture model, so this is a direct, falsifiable prediction about *your* 30 layers: **layers 1–2 should look local (bright diagonal, weak column 0),**

**with the sink appearing from roughly layer 3 onward.** If your layer sweep reproduces that on a 135M model on your laptop, you have a published-paper prediction confirmed live in the demo. If it doesn't, that's worth knowing before you build the guidance panel.

That makes the layer sweep the next thing to run, before I write the overlay.

---

**Solid** — Table 1 confirmed as the location of 5158.07 / 5.40 / 5.60, with the config definitions matching your panel's wording; "we term these tokens 'attention sinks'" confirmed in the Introduction, establishing the priority claim; full author list and affiliations read from the paper header; the ten-model list confirmed in Section 4.1; the BERT/[SEP] finding confirmed in Appendix H; the layers-0-and-1 local pattern confirmed in Figure 2 and Appendix F. All from the arXiv HTML v3 fetched this turn.

**Uncertain / contested** — The representational-collapse clause remains unchecked against its source. The paper's Appendix H claim that sinks are "a universal characteristic across all Transformer models" is the authors' contention, not a survey finding, which is why I'd still avoid "every model anyone has looked at." The ViT "registers" parallel is Darcet et al.'s work, characterized here secondhand. Whether the layers-0-and-1 prediction transfers from a 7B model to a 135M one is exactly what's untested.

**To verify independently** — Run the layer sweep at layers 1, 2, 3, 6, 12, 18, 24, 30 on mean-of-9 and record the column-0 ratio at each; that single table settles whether the paper's depth prediction holds on SmolLM2 and tells us what the overlay should emphasize.

## YOU

I'll have to take a break. My brain is getting overloaded. I'm worried I won't perform these tests correctly.

Even with the default prompt, will I be able to test other prompts and all text and display match accordingly?

**CLAUDE**

Take the break. Nothing here is time-sensitive, and the tool works today.

Short answer to your question: **yes for sections 1, 2, 4, 5 and 6 — they're all computed from whatever prompt you type. But there's one real footgun, and it's worth knowing before you demo.**

**The footgun**

Changing the prompt and clicking **Predict** refreshes the token chips and the distribution. It does **not** clear sections 4, 5 or 6. `runPredict()` resets the rank result and the dart tally and nothing else. `attnCache` is only ever overwritten when you click **Load attention** again.

So: demo prompt A, run the lenses, load attention, then type prompt B and hit Predict — and the lens table and attention grid will still be showing prompt A's results, with no warning. In a live demo that's exactly the kind of thing a skeptical coworker would catch and you wouldn't.

The fix is small: clear those three panels inside `runPredict()`. Five lines. I'll do it whenever you're ready, and I'd do it before anything else.

**Two panel sentences that assume the default prompt**

Both hedged, so not wrong, but they'll read oddly on another prompt:

- The tokens panel explains " os" and " sh" as the model spelling *Osaka* or *Shizuoka*. Only true for the Japan prompt.
- The residual-stream panel says `|h|` grows from about 2.5 to over 700 by layer 29. That was measured on the default prompt; other prompts will differ somewhat.

Neither breaks anything. Just don't read them aloud while a different prompt is on screen.

Everything else adapts: tokenization, all 49,152 logits, the number line, generation, the trace, both lens columns, the attention grids and the bottom-row summary. The only hard limit is that attention truncates at 32 tokens, and the page tells you when it does.

**On the tests**

You can't do them wrong. It's a dropdown. Click **Load attention** once, then change **Layer** from 1 to 2 to 3 and so on, reading the leftmost column each time. No commands, no files, nothing that

can break — switching layers doesn't even hit the server. If the numbers look strange, paste them and we'll work it out together.

And if the layer sweep never happens, the tool is still finished. That test is for a panel that's a nice-to-have.

---

**Solid** — `runPredict()` clears only `rankResult` and `tally`; `attnCache` is assigned only inside `loadAttention()`; `lensDiv` and `traceDiv` are cleared only by their own handlers. Read from your file just now.

**Uncertain** — Whether the `|h|` figures drift enough on other prompts to look wrong, or just slightly off. Untested.

**YOU**

Is this ok?

CLAUDE

Yes — that's exactly right, and you found something better than what I asked you to look for.

Look at the bottom row across the four heads:

| head      | attention on The (col o) | vs uniform 16.7% |
|-----------|--------------------------|------------------|
| 1         | 30.3%                    | 1.82×            |
| 2         | 18.7%                    | 1.12×            |
| 3         | 95.6%                    | 5.74×            |
| 4         | 9.2%                     | 0.55×            |
| mean of 9 | 65.4%                    | 3.92×            |

Head 3 is a near-pure sink — 95.6% of its budget on a word it has no reason to care about. Head 4 goes the other way: 9.2% is *below* what uniform attention would give, so it's actively steering away from position o, and it's reading *is* and *called* instead. That's a local/recency head.

The 65.4% in the mean-of-9 grid isn't "every head puts about two-thirds on The." It's an average over heads ranging from 0.55× to 5.74×. That's the "the mean smooths away specialization" claim in the panel, demonstrated on your own model in four screenshots — no longer an assertion you're asking people to take on faith.

A free consistency check for later

If the mean is 65.4% and your four heads sum to 153.8, the other five must sum to 434.8 — averaging **87% each**. So heads 5 through 9 should be mostly sink-heavy, like head 3.

That's a falsifiable prediction from your own data. Four more dropdown clicks whenever you feel like it. If they come in around 87% on average, the tool is internally consistent and you've got a clean story. If they don't, something's wrong with the mean calculation and I'd want to know.

But that's for later. You've already got the finding — go take the break.

**Solid** — The four bottom-row percentages are read directly from your screenshots' summary lines; ratios and the 87% implication recomputed here from those four values plus the 65.4% mean.

**Uncertain** — The 87% figure assumes the mean-of-9 is a plain unweighted average of the nine head grids, which is what the code does but which the check itself would confirm. Head 4's 9.2% is the summary-line figure; the grid cell prints blank because it's under the 10% display threshold, so nothing is missing.

**YOU**

Here is 9:

**CLAUDE**

That's the cleanest one yet. Head 9 is a pure sink head — every single row, 94% to 100% on The , and essentially nothing anywhere else.

Row by row against its own baseline: 1.00×, 1.96×, 2.91×, 3.76×, 4.80×, 5.82×. The ratio climbs steadily while the raw number barely moves. This head isn't reading the sentence at all. It found position 0 and parks its entire budget there regardless of which token is asking.

Put head 9 and head 4 side by side and you have the whole argument in two images. Same layer, same prompt, same six tokens. One head puts 97.3% on the first word; the other puts 9.2% — below what random uniform attention would give — and reads the recent tokens instead. Nine heads, doing visibly different jobs.

Updated accounting: five heads known, summing to 251.1. For the mean to land at 65.4%, heads 5–8 must sum to 337.5, averaging **84.4%** each. Four clicks, whenever.

Now go rest. This was a good session — you fixed two panels, verified a paper, and found the head that makes the point.

---

**Solid** — Head 9's per-row ratios computed from the six printed column-0 values against  $1/(i+1)$ ; the 84.4% figure follows from the five known bottom-row values and the 65.4% mean.

**Uncertain** — Head 9's off-column cells are all below the 10% print threshold, so I know they're small but not their exact values. Hovering would give them if you ever want the row to sum visibly to 100.

**YOU**

Ok. Can you start with the next step? I'll catch up when I get back.

**CLAUDE**

He's away, so I'll do the whole batch: the stale-panel fix, the two display features, and the sentence corrections. Let me look at the current code first.

**YOU**

Load attention Layer 30 Head mean of all 9 Scale × uniform The default view — layer 30, mean of all 9 — is the worst place to look for meaning, because averaging hides the differences between heads. Step through individual heads at a fixed layer, or pick max of all 9. Then switch Scale to × uniform to see which cells are actually above what uniform attention would give. The Japan → France swap is worth trying head by head; whether any head visibly reroutes is something to check rather than assume. 30 layers x 9 query heads = 270 grids, all loaded. Switching between them costs no server request. The cow ran to the The cow 1.6 0.4 ran 2.0 0.2 to 2.7 0.2 0.4 the 3.1 0.2 0.1 0.3 3.1 0.1 0.0 0.1 Rows sum to 1. Grey cells are causally masked - those values are exactly zero. Every cell is divided by the uniform baseline for its row,  $1/(i+1)$ . 1.0x is exactly what uniform attention would give. Blue is above uniform, amber below. Numbers shown at 1.5x and above, or 0.6x and below. Hover any cell for the exact figure. Bottom row (the position that predicts the next token) is looking at: The #0 (52.4%, 3.1x uniform), the #4 (22.1%, 1.3x uniform), #5 (21.4%, 1.3x uniform). Column 0 as a multiple of uniform: row 1 1.6x, row 2 2.0x, row 3 2.7x, row 4 3.1x, row 5 3.1x. Row 0 is omitted: it has one unmasked cell, so it is 100% by arithmetic and says nothing. Rising numbers mean the sink strengthens as the row gains alternatives to choose from, even when the raw percentage falls.

What do you think? I think it's good

CLAUDE

It works, and the more interesting result is that the sink reproduced on a completely different prompt.

| row | "The capital of Japan is called" | "The cow ran to the " |
|-----|----------------------------------|-----------------------|
| 1   | 1.58×                            | 1.6×                  |
| 2   | 2.25×                            | 2.0×                  |
| 3   | 2.64×                            | 2.7×                  |
| 4   | 3.10×                            | 3.1×                  |
| 5   | 3.90×                            | 3.1×                  |

Same monotone climb, near-identical magnitudes, nothing in common between the sentences. That's the strongest evidence you've generated so far that the sink is structural rather than about content — and it directly answers the question you asked before the break. Other prompts work, and the phenomenon travels.

That said, three things I'd want fixed before a coworker sees this.

1. A contradiction I introduced

The note still opens with "Rows sum to 1." next to a grid whose rows visibly don't. In × uniform mode the displayed numbers sum to  $i+1$ , not 1. The underlying weights still sum to 1 — but a skeptic reading the screen sees a claim contradicted by the numbers beside it. My error; it should read something like "the underlying weights sum to 1 per row; the multiples shown here sum to  $i+1$ ."

Row 0 is also fully blank in ratio mode, which reads as broken rather than as "exactly uniform." Printing a grey 1.0 would say what's meant.

2. A pre-existing bug this prompt exposed

Your dart tally shows:

[all others] -0.00%

`tailExp` starts at 1 and subtracts the top 8 probabilities. At  $T = 0.05$  those sum to 1.0 within float error, so the remainder lands microscopically below zero. It's a rounding artifact, not a real negative, but "negative probability" on screen is exactly the kind of thing that costs you the room. One-line fix: clamp at zero.

### 3. A demo hazard, not a bug

Your prompt ends with a trailing space, so it tokenizes as `The / cow / ran / to / the / " "` — the space becomes its own token. The model's response to that is to predict **digits**: ranks 2 through 10 are `1 , 2 , 4 , 3 , 5 , 7 , 6 , plus urn .` To a skeptical coworker, section 2 looks like the model has lost the plot.

It hasn't. A dangling space is a strong signal of list or table context in training data, and the model is behaving sensibly given a weird input. But I'd avoid trailing spaces in the live demo unless you specifically want to show that tokenization is unforgiving — in which case it's a great example, deliberately chosen rather than stumbled into.

### Worth noting about the J-lens

On this prompt the J-lens is much worse than on the Japan one: 28 of 31 stages disagree, and it ranks the eventual answer around #49,000 through the early layers while producing tokens like `firehose` and `coals`. It only becomes useful around layer 21.

The stage-30 diagnostic still passes at  $3.481e-5$ , so the lens isn't broken — this is the twelve-prompt-corpus limitation your own caution panel warns about, showing up in practice. Good honesty material, but don't lead with the J-lens on an off-corpus prompt.

Want me to do the three fixes? Roughly ten lines total.

---

**Solid** — Ratio values in your grid are consistent with the raw percentages (row 5 col 0:  $52.4\% \times 6 = 3.14\times$ ; row 4 self:  $27\% \times 5 = 1.35\times$ , correctly suppressed as between 0.6 and 1.5). Row 0 blanking, the `-0.00%` arithmetic, and the "Rows sum to 1" string are all traceable to specific lines I can point at.

**Uncertain / contested** — My explanation for *why* a trailing space elicits digits is plausible-inference, not verified. The cross-prompt ratio agreement is two prompts, not a sample; whether the 1.6→3.1 shape is general or a coincidence of two six-token prompts needs a third and fourth prompt of different lengths.

**To verify independently** — Run a prompt with no trailing space and a different length (say 9 or 10 tokens) and check whether the column-0 ratios still climb monotonically.

## YOU

The horse ate all of the oats , but none of the The horse 1.6 0.4 ate 2.0 0.2 all 2.6 0.2 0.6 of 3.2 0.2 0.4 the 3.6 0.3 0.5 0.2 0.3 oats 3.8 0.3 0.4 0.2 0.2 0.2 2.0 , 4.4 0.3 0.4 0.1 0.1 0.3 but 4.9 0.2 0.3 0.1 0.1 0.1 0.4 2.1 none 5.5 0.2 0.4 0.2 0.1 0.1 of 5.8 0.2 0.5 0.1 0.2 0.2 0.6 0.4 the 6.5 0.3 0.3 0.1 0.1 0.4 0.4 0.2 0.2 0.4 2.0 Rows sum to 1. Grey cells are causally masked - those values are exactly zero. Every cell is divided by the uniform baseline for its row,  $1/(i+1)$ . 1.0x is exactly what uniform attention would give. Blue is above uniform, amber below. Numbers shown at 1.5x and above, or 0.6x and below. Hover any cell for the exact figure. Bottom row (the position that predicts the next token) is looking at: The #0 (54.0%, 6.5x uniform), the #11 (16.4%, 2.0x uniform), oats #6 (9.3%, 1.1x uniform). Column 0 as a multiple of uniform: row 1 1.6x, row 2 2.0x, row 3 2.6x, row 4 3.2x, row 5 3.6x, row 6 3.8x, row 7 4.4x, row 8 4.9x, row 9 5.5x, row 10 5.8x, row 11 6.5x. Row 0 is omitted: it has one unmasked cell, so it is 100% by arithmetic and says nothing. Rising numbers mean the sink strengthens as the row gains alternatives to choose from, even when the raw percentage falls.

CLAUDE

That's a stronger result than the two six-token prompts, and it changes what the pattern actually is.

Back out the raw percentages from your ratios:

| row | ratio | implied raw % on The |
|-----|-------|----------------------|
| 1   | 1.6×  | 80.0%                |
| 2   | 2.0×  | 66.7%                |
| 3   | 2.6×  | 65.0%                |
| 4   | 3.2×  | 64.0%                |
| 5   | 3.6×  | 60.0%                |
| 6   | 3.8×  | 54.3%                |
| 7   | 4.4×  | 55.0%                |
| 8   | 4.9×  | 54.4%                |
| 9   | 5.5×  | 55.0%                |
| 10  | 5.8×  | 52.7%                |
| 11  | 6.5×  | 54.2%                |

The raw share **decays and then flattens at about 54%** — rows 6 through 11 sit at 54.3, 55.0, 54.4, 55.0, 52.7, 54.2, mean 54.3%, spread under 2.5 points across six rows of steadily growing context.

So the ratio isn't climbing because the sink is strengthening. It's climbing because the sink holds a roughly **constant fraction of the budget** while the uniform baseline keeps shrinking. Once you're past about row 5, the model reserves ~54% for position 0 no matter how many alternatives it has, and  $\text{ratio} \approx 0.54 \times (i+1)$  almost exactly.

That's a cleaner claim than "the sink strengthens," and it's testable in one more click: a longer prompt should keep the raw share near 54% while the ratio keeps climbing linearly. If you run 20 tokens and row 19 reads about 10.5×, the constant-fraction reading is confirmed.

Which means **my note text is wrong** and needs fixing along with the other three items. "Rising numbers mean the sink strengthens as the row gains alternatives" is the natural reading and the wrong one.

## Two other things in this grid

**The diagonal is real and consistent.** `oats` → itself 2.0×, but → itself 2.1×, final the → itself 2.0×. Three separate rows landing at almost exactly 2× self-attention.

**This prompt has a right answer, and the mean nearly hides it.** "The horse ate all of the oats, but none of the \_\_\_\_" should route back to `oats`. The bottom row does reach it — 9.3%, 1.1× — but that's barely above uniform in the mean of 9.

This is the Japan→France test done properly, because unlike Japan this prompt has something the model actually needs to retrieve. Switch **Head** to **max of all 9** and look at the `oats` column in the bottom row. If some single head is doing retrieval, the max will show it well above 1.1× while the mean buries it. If the max also comes back near 1×, then no head at layer 30 is doing that job and it's worth checking layers 12–20.

Either answer is a good demo. One says "look, specialization." The other says "the honest version of this tool tells you when there's nothing there."

**Solid** — Raw percentages recomputed from your ratios and the  $1/(i+1)$  baseline; rows 6–11 average 54.3% with a 2.3-point spread; ratios strictly monotone increasing across all 11 rows; bottom-row cross-check  $54.0\% \times 12 = 6.48$ , matching the displayed 6.5×.

**Uncertain / contested** — The implied raw percentages carry rounding error from your one-decimal ratios, roughly  $\pm 0.5$  points at row 1 and  $\pm 0.4$  at row 11, so "54.3%" is really "somewhere around 54%." Whether the plateau holds beyond 12 tokens is untested — that's the one input most affecting whether I rewrite the note as "constant fraction" or something more hedged. Three prompts is not a sample.

**To verify independently** — Run a 20-token prompt and check whether row 19's ratio lands near 10.5× and the implied raw share stays near 54%.

**YOU**

Uniform:

The default view — layer 30, mean of all 9 — is the worst place to look for meaning, because averaging hides the differences between heads. Step through individual heads at a fixed layer, or pick max of all 9. Then switch Scale to  $\times$  uniform to see which cells are actually above what uniform attention would give. The Japan  $\rightarrow$  France swap is worth trying head by head; whether any head visibly reroutes is something to check rather than assume. 30 layers  $\times$  9 query heads = 270 grids, all loaded. Switching between them costs no server request. The horse ate all of the oats, but none of the hay. I'm taking a wild guess that it doesn't like The horse 1.6 0.4 ate 2.0 0.2 all 2.6 0.2 0.6 of 3.2 0.2 0.4 the 3.6 0.3 0.5 0.2 0.3 oats 3.8 0.3 0.4 0.2 0.2 0.2 2.0, 4.4 0.3 0.4 0.1 0.1 0.3 but 4.9 0.2 0.3 0.1 0.1 0.1 0.4 2.1 none 5.5 0.2 0.4 0.2 0.1 0.1 of 5.8 0.2 0.5 0.1 0.2 0.2 0.6 0.4 the 6.5 0.3 0.3 0.1 0.1 0.4 0.4 0.2 0.2 0.4 2.0 hay 6.4 0.3 0.3 0.1 0.1 0.1 0.3 0.4 0.2 0.2 3.2. 6.9 0.2 0.2 0.0 0.0 0.0 0.2 0.2 0.5 0.0 0.0 0.1 0.4 5.1 6.2 0.2 0.2 0.0 0.0 0.0 0.2 0.1 0.3 0.0 0.0 0.0 0.2 4.4 3.0 I 10.0 0.4 0.2 0.1 0.0 0.0 0.2 0.2 0.1 0.1 0.0 0.0 0.3 0.5 2.8 'm 11.3 0.3 0.1 0.0 0.0 0.0 0.1 0.1 0.1 0.1 0.0 0.1 0.1 0.3 0.3 1.7 2.3 taking 7.8 0.3 0.3 0.1 0.1 0.2 0.3 0.5 0.1 0.1 0.2 2.2 0.4 0.3 0.4 3.7 a 10.0 0.4 0.3 0.1 0.0 0.3 0.2 0.1 0.1 0.0 0.3 0.3 0.2 0.3 0.3 1.8 3.0 wild 11.7 0.4 0.2 0.0 0.0 0.1 0.6 0.2 0.1 0.0 0.0 0.2 0.3 0.2 0.2 0.3 0.1 2.1 2.5 guess 9.8 0.2 0.2 0.1 0.0 0.0 0.4 0.2 0.3 0.1 0.0 0.0 0.3 1.5 0.4 0.2 0.5 4.0 that 8.5 0.4 0.3 0.1 0.1 0.1 0.3 0.1 0.1 0.2 2.8 0.2 0.1 0.4 0.1 0.2 0.5 4.4 it 12.4 0.5 0.4 0.1 0.1 0.1 0.2 0.2 0.2 0.1 0.1 0.2 0.4 0.1 0.2 0.1 0.1 0.3 3.7 doesn't 17.3 0.1 0.0 0.0 0.0 0.0 0.1 0.0 0.0 0.0 0.0 0.0 0.1 0.1 0.3 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.4 3.4 't 11.9 0.3 0.5 0.2 0.1 0.1 0.2 0.2 0.2 0.1 0.1 0.3 1.6 0.3 0.1 0.2 0.1 0.2 0.4 3.1 0.4 2.1 like 10.0 0.4 0.4 0.2 0.1 0.5 0.3 0.1 0.1 0.5 1.6 0.5 0.2 0.1 0.4 0.1 0.5 0.1 0.5 0.3 0.1 0.3 5.9 Rows sum to 1. Grey cells are causally masked - those values are exactly zero. Every cell is divided by the uniform baseline for its row,  $1/(i+1)$ . 1.0x is exactly what uniform attention would give. Blue is above uniform, amber below. Numbers shown at 1.5x and above, or 0.6x and below. Hover any cell for the exact figure. Bottom row (the position that predicts the next token) is looking at: The #0 (38.4%, 10.0x uniform), like #25 (22.8%, 5.9x uniform), hay #12 (6.0%, 1.6x uniform). Column 0 as a multiple of uniform: row 1 1.6x, row 2 2.0x, row 3 2.6x, row 4 3.2x, row 5 3.6x, row 6 3.8x, row 7 4.4x, row 8 4.9x, row 9 5.5x, row 10 5.8x, row 11 6.5x, row 12 6.4x, row 13 6.9x, row 14 6.2x, row 15 10.0x, row 16 11.3x, row 17 7.8x, row 18 10.0x, row 19 11.7x, row 20 9.8x, row 21 8.5x, row 22 12.4x, row 23 17.3x, row 24 11.9x, row 25 10.0x. Row 0 is omitted: it has one unmasked cell, so it is 100% by arithmetic and says nothing. Rising numbers mean the sink strengthens as the row gains alternatives to choose from, even when the raw percentage falls.

And Raw:

The default view — layer 30, mean of all 9 — is the worst place to look for meaning, because averaging hides the differences between heads. Step through individual heads at a fixed layer, or pick max of all 9. Then switch Scale to  $\times$  uniform to see which cells are actually above what uniform attention would give. The Japan  $\rightarrow$  France swap is worth trying head by head; whether any head visibly reroutes is something to check rather than assume. 30 layers  $\times$  9 query heads = 270 grids, all loaded. Switching between them costs no server request. The horse ate all of the oats, but none of the hay. I'm taking a wild guess that it doesn't like The 100 horse 82 18 ate 67 25 all 64 17 14 of 63 13 12 the 59 19 oats 54 29, 55 13 18 but 54 24 none 55 12 10 of 52 11 the 54 16 hay 49 24. 50 36 41 29 20 I 63 17 'm 66 10 14 taking 43 12 20 a 53 16 wild 58 10 12 guess 47 19 that 39 13 20 it 54 16 doesn't 72 14 't 48 12 like 38 23

CLAUDE

This falsifies what I told you last turn. Good — that's what the longer prompt was for.

My "constant ~54%" claim doesn't survive

Column o, raw share, all 26 rows:

| rows  | n  | mean  | sd   | range |
|-------|----|-------|------|-------|
| 6–13  | 8  | 52.9% | 2.1  | 49–55 |
| 14–25 | 12 | 51.8% | 10.7 | 38–72 |
| 6–25  | 20 | 52.2% | 8.4  | 38–72 |

The *level* holds up — about 52% on average, close to the 54% I quoted. The *stability* does not. Standard deviation goes from 2.1 to 10.7 the moment the prompt runs past twelve tokens. Row 23 ( doesn ) takes 72%; row 25 ( like ) takes 38%.

I saw rows 6–11 of your twelve-token prompt and called it a plateau. Rows 6–13 *are* a plateau — sd 2.1, remarkably tight. It just doesn't continue. I generalized from eight rows of one prompt, which is exactly the small-retrospective-sample error, and I made it one turn after telling you three prompts wasn't a sample.

**The ratio isn't even monotone.** It falls at 7 of the 24 transitions: 6.5→6.4, 7.0→6.2, 11.2→7.7, 11.6→9.9, 9.9→8.6, 17.3→12.0, 12.0→9.9.

Corrected version: the sink holds roughly half the attention budget regardless of how long the context gets, varying ±10 points row to row once the sentence gets long. The ratio trends upward because the baseline shrinks, not because the sink grows. That's still a strong claim — a fixed ~50% reserve on a token that carries no information is the whole point — it's just not the tidy monotone story.

My note text is now wrong twice

Rising numbers mean the sink strengthens as the row gains alternatives to choose from  
They don't always rise, and rising wouldn't mean strengthening. That line needs to go.

The finding I'd actually build the demo around

Scan the ratio grid. Above 1.5× you have column 0, and the diagonal — oats → itself 2.0×, but 2.1×, the 2.0×, hay 3.2×, . 5.1×, and so on up to like at 5.9×. Almost everything else is amber. Twenty-six tokens, nine heads averaged, and essentially all above-uniform attention sits on *position 0* and *self*.

That's a sharper statement than anything about the sink alone: at layer 30, the mean-of-9 view shows the model doing almost no visible content routing. Which is either evidence that individual heads are where the routing lives, or evidence that layer 30 isn't where it happens. Max-across-heads and a middle layer distinguish those.

## The fix list

Four items now: the false "Rows sum to 1" in ratio mode, blank row 0, the wrong interpretation sentence, and the -0.00% dart tally. Want me to do them?

---

**Solid** — All 26 raw column-0 values read from your raw grid; ratios recomputed as  $\text{row} \times (i+1)$  and matching your ratio grid throughout (row 23:  $72 \times 24 = 17.3\times$ ; row 14:  $41 \times 15 = 6.2\times$ ); monotonicity tested and failed at seven transitions; segment statistics computed here.

**Uncertain / contested** — Off-diagonal cells are hard to align reliably in a pasted table, particularly around the blank token at index 14, so treat my specific off-diagonal readings as tentative; the column-0 and diagonal claims are safe because their positions are unambiguous. Whether the high-sink rows ( I , 'm , doesn at 63–72%) share a linguistic property is pure speculation — I have four data points and no hypothesis worth the name.

**To verify independently** — Switch to max of all 9 on this prompt and check whether any off-diagonal cell clears 3×. If none does, layer 30 genuinely isn't doing content routing and the middle layers are where to look.

**YOU**

**CLAUDE**

Those two views side by side show two rendering problems I created. Fixing them along with the other four.

**YOU**

**CLAUDE**

Both fixes landed visibly. Row 0 reads 1.0, and the wash is gone — image 2 is now mostly white with the structure standing out instead of drowning in tint.

**One thing the fix created**

Masked cells and blanked cells now look nearly the same. The causal mask renders #f4f4f4; a sub-0.25× cell renders transparent white. On screen those are close to indistinguishable, and the mask is a teaching point — "those values are mathematically zero, not just small" is a claim the grid should support visually. Right now it doesn't.

Small CSS fix: darken td.masked a shade and give it a faint diagonal hatch so "impossible" and "negligible" read differently. Want that?

**What's now visible that wasn't**

Read down the rightmost printed cell in each row — that's self-attention:

Early rows: nothing. horse gives itself 0.4×, ate and all fall below the print threshold entirely.

From oats onward it turns on and grows: 2.0, 2.1, 2.0, 3.2, 5.1, 3.0, 2.8, 2.3, 3.7, 3.0, 2.5, 4.0, 4.4, 3.7, 3.4, 2.1, and like at 5.9×.

So in the mean-of-9 at layer 30, essentially all above-uniform attention sits on **position 0** and **self**, and the self component is absent for the first several tokens then settles around 3×. Two structural patterns, no visible content routing anywhere in a 26-token sentence with an obvious referent in it.

That's a cleaner statement of the sink panel's thesis than the panel makes: bright cells are telling you about position, not meaning. You now have a 26×26 grid demonstrating it.

Which makes **max of all 9 on this prompt** the test worth running. If content routing exists at layer 30, it's hiding in a single head. If the max grid also shows only column 0 and the diagonal, then layer 30 isn't where the model does that work, and the middle layers are the next place to look.

---

**Solid** — Row 0 renders 1.0; the sub-0.25× blanking is working (large white regions where image 2 previously showed a continuous tint); the rightmost printed value in each row is unambiguously the diagonal, and those values run 2.0–5.9× from oats onward.

**Uncertain / contested** — I'm reading cell positions off a screenshot, so any claim about *interior* off-diagonal cells is unreliable; column 0 and the rightmost cell are the only positions I can locate with confidence. Whether early-row self-attention is genuinely absent or merely crowded out by an 80% sink share is not distinguishable from this grid.

**To verify independently** — Switch to max of all 9 and check whether any interior cell clears 3×.

YOU

CLAUDE

That answers it — and it flips the conclusion from two turns ago. Content routing does exist at layer 30. It was entirely hidden by the mean.

Bottom row, like , comparing the two aggregations:

| view      | above-uniform cells besides column 0 and self |
|-----------|-----------------------------------------------|
| mean of 9 | one ( hay , 1.6×)                             |
| max of 9  | at least five — 8.6×, 5.4×, 3.0×, 2.8×, 2.2×  |

The interior of the max grid is full of values the mean showed nothing for: 19.2×, 17.6×, 15.0×, 14.1×, 13.7×, 13.6×, 12.0×, 11.7×, 11.5×, 11.2×, 11.0×. My test threshold was "does anything clear 3×." Plenty clears ten.

Column 0 in the max view is 79–100% raw on *every* row. So for each row there is some head putting nearly its entire budget on position 0 — consistent with head 3 and head 9 behaving that way on the shorter prompt. And you can see previous-token structure directly: 'm reaching back to I at 43%, hay to the preceding the at 31%.

But before you show this to anyone, one caveat, and I'd rather raise it than have a coworker do it.

**YOU**

Oats was on 11, and hay was on 21 - 33 was "like"

**CLAUDE**

Hold on — that doesn't reconcile, and it matters.

In the raw max grid, the bottom row prints six values: 79, 11, 21, 33, 11, 68. The **68** is the rightmost cell in the row, which is column 25 — `like` itself. Two cells can't both be `like`.

So either I'm misreading the image, or the tooltip you hovered said something other than what it looked like. My guess is the 33 sits at the blank column, index 14 — your prompt has "hay." then a line break before "I'm", so token 14 is probably a newline. Its tooltip would render as `like -> : 33.00%` with nothing visible after the arrow, which is easy to misparse.

Could you hover that cell and paste the tooltip verbatim? It changes the story: a newline pulling 33% is a *second sink*, which is a different and also interesting finding from a content reach-back.

### **What your two confirmed numbers say**

Running the multiple-comparison version of the null — the largest of 25 interior cells, not a single cell:

**YOU**

Here's all from the last row

CLAUDE

Confirmed, and the 33 mystery is solved: like -> : 32.99% — the tooltip has nothing after the arrow. That's position #14, and the summary line prints it as #14 (33.0%, 8.6x uniform) with no token text at all.

Your five hovers, converted:

| target          | raw   | × uniform | verdict                           |
|-----------------|-------|-----------|-----------------------------------|
| The #0          | 78.8% | 20.5×     | sink                              |
| like #25 (self) | 67.8% | 17.6×     | diagonal                          |
| #14 (invisible) | 33.0% | 8.6×      | beyond anything the null produced |
| hay #12         | 20.9% | 5.4×      | beyond the null                   |
| oats #6         | 11.4% | 3.0×      | noise band                        |
| wild #19        | 10.7% | 2.8×      | noise band                        |

So hay holds up, oats and wild don't, and the second-strongest thing the final position looks at — after the sink and itself — is **a token with no visible text**. Almost certainly the newline between hay. and I'm , since I tokenized without its usual leading space.

That's the best result of the session, and it connects directly to the paper you just read. StreamingLLM found that replacing the four initial sink tokens with plain line-break tokens restored perplexity to 5.60 against 5.40 for the real tokens. Your model, unprompted, is treating a line break as a major attractor. Same phenomenon, your hardware, live.

It also exposes a real bug: a whitespace token is **invisible everywhere** — blank column header, blank row label, blank tooltip, blank in the summary. You only found it by elimination. Fixing that plus the two outstanding items.

YOU

I think this is 14?

**CLAUDE**

Both dots on screen are different tokens and my glyph choice hid that. · (space) at 26px is nearly indistinguishable from . (period), and the tooltip reads · -> · with no positions — so it can't answer your question. That's my bug, not your misreading.

What the tooltip *does* pin down: the baseline is 3.7%, and  $1/27 = 3.70\%$ , so you're on **row 26** of a 27-token prompt. You've added a trailing space since the last run — the summary now ends with · #26 (18.2%, 4.9x uniform) .

Fixing the tooltip to carry positions, and switching to a glyph that can't be confused with punctuation.

**YOU**

**CLAUDE**

position 14: " " — a plain space. Not a newline.

I claimed newline twice, first as inference and then more confidently than the evidence supported. It was wrong both times. The header tooltip settles it, which is what that fix was for.

## What the finding actually is

On the 26-token prompt, in max-across-heads view, the final position's second-largest target after the sink and itself was **a lone space token** at 33.0% — 8.6× uniform, well beyond anything the null simulation produced.

That's a stronger demo point than a newline would have been. A newline at least marks a paragraph break; a bare space marks nothing at all. Some head at layer 30 is putting a third of its budget on a character that carries no information whatsoever.

And it lines up with the paper you read this morning. StreamingLLM found the four initial sink tokens could be swapped for plain line-break tokens with perplexity going from 5.40 to 5.60 — semantically empty tokens work as sinks. Your model, on your laptop, spontaneously recruited a semantically empty token as a secondary sink. Same phenomenon, arrived at independently.

## One thing to check before you show it

A standalone " " token between . and I is unusual — BPE would normally produce " I" as a single token. That suggests you typed **two spaces** after hay.

Worth knowing which it is, because a coworker may notice. If it's a double space, the honest framing gets better, not worse: an accidental extra keystroke created a token the model then leaned on heavily. That's a sharper illustration of "tokenization is unforgiving" than anything the panels currently say.

The test: retype the prompt with a single space after the period and reload. If the standalone space disappears from the token strip, it was your keystroke. Then check whether the . at position 13 inherits the secondary-sink role — if it does, the model wants *something* structural to lean on and will take whatever is available. If nothing inherits it, the space was doing something specific.

Also: you still have a trailing space at #26 from the last edit. Worth removing before the demo unless you want to talk about it.

**Solid** — Position 14 is " ", read directly from the header tooltip; the \_ glyph and header titles both render correctly, so that patch works as intended; the 33.0% / 8.6× figure was measured on the 26-token prompt in max view.

**Uncertain / contested** — Whether the standalone space came from double-spacing or from tokenizer behaviour; I'm inferring from BPE conventions, which is the same kind of reasoning that just produced a wrong answer about the newline. Whether the . would inherit the role is untested.

**To verify independently** — Retype with a single space, check whether the standalone space token survives, and if not, re-run max view and see what position 13 does.

**YOU**

**CLAUDE**

Position 14 is " " — confirmed a second time, and the header tooltips are working, which is what I needed them for.

The position 26 tooltip is clipped by the right edge of the window. Native browser tooltips position near the cursor and won't reflow for the last column, so for the rightmost few columns you'll want to scroll left a bit before hovering, or read that token off the chip strip in section 1 instead.

**Settling the double-space question**

One hover does it: **the header at position 15.**

- If it reads position 15: " I" — leading space — then you typed two spaces after hay. The tokenizer took one as a standalone token and gave the other to I .
- If it reads position 15: "I" — no leading space — then a single space became its own token, which would be unusual for byte-level BPE and worth knowing about on its own.

That's the last open thread from this thread of investigation, and either answer is usable in the demo.

---

**Solid** — Position 14 is " ", read from the header tooltip; header tooltips render and carry the JSON string as intended.

**Uncertain / contested** — Whether the standalone space came from double-spacing. Position 26's token, since the tooltip is cut off — though a trailing space is the obvious candidate given you added one.

**To verify independently** — Hover the header at position 15.

**YOU**

**CLAUDE**

Clipped again — I can see position 15: " and then it runs off the edge. I'm not going to squint at the gap after the quote and call it, since that's exactly the move that got the newline wrong.

Definitive check that can't be cut off, straight in a browser tab:

```
http://127.0.0.1:8000/api/tokenize?
prompt=The%20horse%20ate%20all%20of%20the%20oats,%20but%20none%20of%20the%20hay.%20%20I'm%20taking%20a%20v
```



(adjust host and port to whatever unicorn is serving). Note the %20%20 after hay. — that's the double space as I think you typed it. The JSON returns exact text for every position, quotes and all. Position 15 will read " I" or "I" with no ambiguity.

You can also just paste the prompt normally in the address bar and let the browser encode it; the double space is the only part worth being careful about.

## A related limitation worth knowing

tokenLabel only rewrites tokens that are *entirely* whitespace. A token like " I" still renders as I with the leading space invisible — in the chips, the headers, everywhere. So the tool can show you a standalone space but not a leading one.

I left it that way deliberately because section 1's panel makes a point of " capital" versus "capital", and I didn't want to change how those render without asking. But the argument cuts the other way too: showing \_capital would make the distinction visible instead of asking people to take it on faith.

That's a real design call, and it touches every token display in the tool. Your demo, your call — worth deciding before you show it to anyone, not after.

---

**Solid** — The tooltip is clipped at the window edge in this image; position 15's header renders as I , which is consistent with either "I" or " I" and therefore settles nothing.

**Uncertain / contested** — Whether position 15 carries a leading space, and so whether the standalone space at 14 came from double-spacing.

**To verify independently** — The /api/tokenize URL above.

**YOU**

## CLAUDE

position 15: " I" — clear gap between the quote and the I . Double space confirmed.

So you typed two spaces after hay . The tokenizer gave one of them to I as the usual leading space, and the other had nowhere to go, so it became a standalone token at position 14. Not tokenizer weirdness. Your keystroke.

Which makes the finding better than I thought two turns ago:

**An invisible typo created a token. A head at layer 30 then put 33% of its budget on it — 8.6× uniform, beyond anything the null simulation produced in 20,000 rows.**

Nobody typed that space meaning anything by it. Nobody could see it on screen until we fixed the renderer. It carries no information at all. And the model reached for it anyway, harder than it reached for hay , which is the word the sentence is actually about.

That's the whole thesis of your sink panel, produced by accident, on your laptop, with a number attached. It's a better artifact than the Japan→France test you originally planned, because it wasn't designed — it was found.

## If you want to finish the story later

Retype with a single space and re-run max view. Either the . at position 13 inherits the secondary-sink role, which says the model wants *something* structural to lean on and will take whatever's nearest, or nothing inherits it, which says that particular space was doing something specific. Both answers are worth having. Neither is urgent.

## Where things stand

Today you fixed the two panels that had been stranded in the wrong chat thread, verified the StreamingLLM paper directly rather than trusting my summary, and added four things to the tool: the uniform-baseline scale, max-across-heads, the stale-panel guard, and visible whitespace tokens. Every one of those came out of an actual question rather than a feature list.

You also caught me being wrong three times — the constant-54% claim, the newline, and my 3× noise threshold — each time by running one more prompt. That's the tool working on its author, which is roughly what you built it for.

**Solid** — Position 15 is " I" with a leading space, read directly from the header tooltip; combined with position 14 being " ", the double space is established. The 33.0% / 8.6× figure was measured on the 26-token prompt in max-across-heads view and exceeds the simulated null maximum.

**Uncertain / contested** — Whether the sink role transfers to . under single spacing. Whether any of this generalizes past layer 30 — every attention finding today is from one layer.

**To verify independently** — Single-space version, max view, check position 13.

**YOU**

**CLAUDE**

Double space confirmed from the other direction: position 13 is "." and position 14 is now " I" . The standalone space is gone. Everything after it shifted down one, so the prompt is 25 tokens instead of 26.

One step left to close this out:

1. Switch **Head** to **max of all 9**
2. Hover the bottom row, column 13 (the . )

Baseline is now  $1/25 = 4.0\%$ , so the thresholds are:

- **Above ~20%** (5× uniform) — the period inherited the role. The model wants something structural to lean on and takes whatever is nearest.
- **Near or below ~12%** (3× uniform) — nothing inherited it. That specific space token was doing something particular, which is the stranger and more interesting result.

For comparison, the space held 33.0% at 8.6× when it existed.

Either way you'll have the answer, and it's the last loose end from today.

**YOU**

**CLAUDE**

#25  -> #13 `.`: 2.47%. With 26 tokens the baseline is  $1/26 = 3.85\%$ , so that's **0.64× uniform — below what uniform attention would give.**

The period did not inherit the role. Not even slightly: the space held 33.0% (8.6×), the period holds 2.47% (0.64×), a factor of about 13 apart.

So it's the stranger branch. The model wasn't looking for *some* structural token to lean on.

Remove the stray space and the secondary sink doesn't relocate to the nearest punctuation — it just isn't there.

**One confound to close**

Your query token changed. The trailing space is still on the end, so #25 is that space and `like` is now #24. Earlier the bottom row *was* `like`. A trailing space asking the question isn't the same as `like` asking it.

Clean comparison, no retyping: hover **row #24, column #13** in the same max view. That's `like` -> `.`, exactly matching the earlier `like` -> . If it also comes back near 0.6×, the finding is solid.

**What you'd have then**

A prompt where an accidental double space created a token that one head at layer 30 gave 8.6× uniform attention, and where deleting that space causes the behaviour to vanish rather than move. The head wanted *that token at that position*, and it existed only because of a typo.

That's a much better artifact than "the model attends to the first token," because a skeptic can watch it appear and disappear under a one-character edit.

**Solid** — Tooltip reads 2.47% for #25  -> #13 `.`;  $1/26 = 3.846\%$ , giving 0.64×; the position tooltip confirms the trailing space is still present at #25 and that the query token differs from the earlier measurement.

**Uncertain / contested** — Whether `like` specifically would also show ~0.6× to the period; the current reading uses a different query token. Everything here remains layer 30 only.

**To verify independently** — Hover row #24, column #13 in max view.

**YOU**

CLAUDE

#24 like -> #13 .: 6.15% . Baseline for row 24 is 1/25 = 4.0%, so **1.54× uniform**.

That's the clean comparison, same query token as before:

| query                            | target      | raw   | × uniform |
|----------------------------------|-------------|-------|-----------|
| like → _ #14 (with double space) | stray space | 33.0% | 8.6×      |
| like → . #13 (single space)      | period      | 6.2%  | 1.54×     |

A factor of 5.6 apart. 1.54× is inside the noise band — the null simulation put the median row-maximum around 2.8×, so a single cell at 1.54× is unremarkable.

**The period did not inherit the role.** Delete the stray space and the secondary sink disappears rather than relocating.

The finding, stated carefully

A head at layer 30 gave 33% of its budget to a token that existed only because of an accidental double space. Remove that keystroke and nothing takes its place. Whatever that head was doing, it was locked to that token at that position — not to "the nearest structural marker," which is what I'd have guessed and what the period test rules out.

I don't have an explanation for it, and I'd rather you present it without one than reach for a story. Anyone can watch it appear and vanish under a one-character edit, which is worth more than a mechanism I'd be inventing.

Two caveats to keep attached

Both comparisons are layer 30 only. And the two prompts differ in length, 26 versus 25 tokens, which shifts every baseline slightly — not enough to explain a 5.6× gap, but it's not a perfectly controlled swap either.

That closes every open thread from today. Good session — you have a working tool, two corrected panels, a verified paper, and a demo artifact you found rather than designed.

**Solid** — Tooltip reads 6.15% for #24 like -> #13 . ; 1/25 = 4.0% gives 1.54×; query token matches the earlier 33.0% measurement, removing the confound from the previous reading; both

measurements are max-across-heads at layer 30.

**Uncertain / contested** — Why that head attached to the space. Whether the effect exists at other layers. The 26-vs-25 token length difference between the two prompts is a small uncontrolled variable.

**To verify independently** — If you want the tightest possible version: build a 26-token single-space prompt by adding one word elsewhere, so both conditions have identical length and only the space differs.

**YOU**

I type with a double space after a period. Could that be the issue?

**CLAUDE**

Yes — that's exactly the cause. Two spaces after `hay .`, the tokenizer gives one to `I` as its usual leading space, and the second has nowhere to go, so it becomes a token of its own.

It's not an error. Double-spacing after a period is a real typographic convention. But it does mean **every prompt you type containing a sentence boundary will carry one of these**, and until an hour ago they were invisible in the tool.

Three practical consequences.

**Nothing from today is contaminated.** Your other prompts — the Japan one, the cow one, the oats/hay one without the second sentence — have no internal periods, so no stray tokens. Only the 26-token prompt had one, and that's the one where we found it.

**Section 1 will now show you.** The token chips use the same `tokenLabel`, so a standalone space renders as `_` in the strip before you ever run the model. You'll see it as you type.

**It's a demo asset, not a liability.** You have a live, one-keystroke demonstration that tokenization is sensitive to characters nobody can see, and that the model's attention changes measurably as a result —  $8.6\times$  down to  $1.5\times$ . The panel in section 1 currently argues that leading spaces matter using `" capital"` versus `"capital"` as an assertion. You can now show it instead. Type the sentence with one space, watch the chip strip, add the second space, watch a `_` chip appear.

The only thing I'd actually change is deliberate: decide before the demo whether you want the double space in your prompt or not, rather than having it there by habit. Both are defensible. Having it there and being surprised by it in front of coworkers is the one bad outcome.

One thing I don't know: whether the model's training data contained much double-spacing. That would bear on why a head latched onto it so hard, and I haven't checked and wouldn't want to guess.

---

**Solid** — The double space is confirmed as the origin of the position-14 token (position 14 was `"`, position 15 was `" I"`); today's other prompts contain no internal periods; `renderTokens` calls `tokenLabel`, so standalone whitespace now renders in the chip strip.

**Uncertain / contested** — Why that head attached to it so strongly; whether double-spacing is rare in SmolLM2's training corpus, which I have not investigated.

**To verify independently** — Type your sentence in section 1 with one space, then two, and watch the chip strip change. That's the demo, and it takes five seconds.

**YOU**

I believe the double space typing was it

**CLAUDE**

Agreed. That's settled by the two tooltips — position 14 " ", position 15 " I" — not by inference.

Worth being precise about what it explains, though. The double space explains **why the token existed**. It doesn't explain **why a head gave it 33% of its budget**, which is still unaccounted for. Deleting the space made the behaviour vanish rather than move to the period, so the head wasn't just grabbing the nearest structural marker. That part remains open, and I'd leave it open rather than let the tokenization answer stand in for it.

For the demo, the two-part version is the strong one: an invisible keystroke created a token, and the model then leaned on it hard. First half is fully explained. Second half you can show without explaining, which is more honest and lands better anyway.

Nothing left open from today that needs action. You have the tool, the panels, the paper, and the artifact.

**YOU**

What's next?!

**CLAUDE**

Four things left, in the order I'd do them.

**1. The layer sweep.** This is the big one and it's still undone. Every attention finding today — the sink ratios, the diagonal, the space token, the `hay` retrieval — is **layer 30 only**. That's one of thirty. StreamingLLM's Figure 2 makes a specific prediction: layers 0 and 1 should look *local*, with recent tokens getting the attention, and the sink appearing from roughly layer 3 onward. If your 135M model reproduces that, you have a published prediction confirmed on your laptop. Ten minutes of dropdown clicking.

**2. The attention guidance panel.** Record §13 item 1, blocked all this time on "someone has actually explored the grids." You've now explored them. After the sweep you'll know what the panel should say, and it'll be written from your measurements rather than my expectations.

**3. Update the project record.** v2 describes none of today: four patches, the corrected panels, the verified paper, the sink findings, the space-token artifact. This session began with a correction stranded in the wrong chat thread because the record didn't capture it. Leaving v2 as-is recreates exactly that failure. I can write v3 whenever you want.

**4. The deliberate failure case,** §13 item 2. Still open, still worth having.

There's also a thirty-second loose end: **heads 5–8** on the short Japan prompt. I predicted they'd average 84.4% on column 0 and you never checked. Pure curiosity, but it's four clicks.

My recommendation: sweep first. It's the largest untested claim in the whole tool, it's cheap, and everything else reads better once it's done.

Layers worth hitting: **1, 2, 3, 6, 12, 18, 24, 30**, mean of 9,  $\times$  uniform. Read the column-0 line at the bottom each time — it gives you every row's multiple in one string, so you don't have to squint at cells.

---

**Solid** — Every attention measurement today is from layer 30; the record on disk is v2 and predates all four patches; §13 items 1, 2 and 4 remain open.

**Uncertain** — Whether the layer-0/1 local pattern transfers from Llama-2-7B to a 135M model. That's the point of running it.

**YOU**

What's the next step that you suggest? BTW, after seeing this in action, I'd like to use this on my UM760 with my razer core on bigger models. I'd love to see the differences. Could it eventually be made to use any model that can be run on my setup?

**CLAUDE**

Next step is unchanged: **the layer sweep**. Ten minutes, and it's the largest untested claim in the tool. Everything else — including this — reads better once it's done.

On the bigger question: yes, and it's a real project rather than a config change. Let me size it honestly.

**YOU**

It's an RTX3060. I'm not worried about loading or processing times.

CLAUDE

Good card for this. Assuming the 12GB desktop variant — the Core is a desktop enclosure so that's near-certain, but worth confirming, since an 8GB card shifts every row below.

What fits

| params | bf16   | 8-bit  | 4-bit  | reachable         |
|--------|--------|--------|--------|-------------------|
| 1.7B   | 3.4 GB | —      | —      | bf16, comfortable |
| 3B     | 6.0 GB | —      | —      | bf16              |
| 7–8B   | 16 GB  | 8.0 GB | 4.4 GB | 8-bit             |
| 14B    | 28 GB  | 14 GB  | 7.7 GB | 4-bit             |

The useful part: **your 32-token attention cap makes activations irrelevant**. Eager attention matrices for an 8B model at 32 tokens are 2 MB, hidden states 9 MB. VRAM is essentially just weights. A tool that streamed 4K contexts would have a completely different budget.

Check this before anything else

Your environment is Python 3.14.7 with torch 2.13.0+cpu. **I don't know whether a CUDA build of torch exists for Python 3.14 on Windows**. 3.14 is recent and PyTorch's Windows CUDA wheels have historically trailed new Python releases. That's a hard gate — if there's no wheel, the answer is a second venv on 3.12, not a code change.

One command tells you:

```
pip index versions torch --index-url https://download.pytorch.org/whl/cu124
```

Don't let me guess this one. I've been wrong twice today on things I inferred instead of checked.

Use bf16, not fp16

Ampere supports bfloat16 natively. Same memory as fp16, much wider exponent range. The logit lens multiplies a residual vector by a  $49k \times$  hidden matrix — at 8B scale with |h| in the hundreds,

fp16 can overflow where bf16 won't. Your `|h|` column already reaches 703 at layer 29 on a 135M model.

And note this breaks some of your on-screen numbers. The causal-mask deviation of  $1.526e-05$  and the J-lens diagnostic threshold of  $1e-3$  were both measured at fp32. At bf16 they'll move by orders of magnitude, and a *correct* J-lens could fail the PASS gate. Those thresholds need to scale with dtype.

Quantization is worse for this: 4-bit changes numerics enough that every exactness claim in the tool needs re-checking rather than re-scaling. I'd stay at bf16 and accept the 3B ceiling before reaching for 4-bit at 14B.

## The first model to port to isn't a big one

**GPT-2 small.** 12 layers, 768 hidden, 12 heads, 50,257 vocab, about 500 MB. Runs on your current CPU setup with no CUDA at all.

It's the right first target because it breaks all the right things:

- `model.model.norm` doesn't exist — it's `transformer.ln_f`. Forces the module resolver immediately.
- LayerNorm instead of RMSNorm, learned positional embeddings instead of RoPE, plain multi-head attention instead of GQA. Your "why 9 heads" panel and the whole GQA explanation stop applying.

That last point is the payoff. Put GPT-2 and SmolLM2 side by side and your coworkers see that "transformer" is a family, not a thing — and that the sink shows up in both anyway, across a seven-year architecture gap. That's a stronger argument than a bigger model of the same design.

Then 1.7B or 3B on the GPU for scale, once the resolver exists.

**Still: sweep first.** It's ten minutes, it's the last untested claim in the current tool, and you don't want to port a curriculum you haven't finished validating.

---

**Solid** — VRAM figures are 2/1/0.55 bytes per parameter, computed here; activation sizes computed from the tool's own 32-token cap; GPT-2 small's config (12 layers, 768 hidden, 12 heads, 50,257 vocab) and its LayerNorm / learned-position / MHA architecture; Ampere supports bf16.

**Uncertain / contested** — Whether torch CUDA wheels exist for Python 3.14 on Windows; I have not checked and will not guess. Whether your 3060 is the 12GB or 8GB variant. Exact layer/hidden configs for specific 1.7B and 3B checkpoints — read those off the model card rather than from me. bitsandbytes on Windows has historically been friction-prone; unverified.

**To verify independently** — The `pip index versions` command above, and your card's VRAM in Task Manager under Performance → GPU.

**YOU**

Maybe we can work on conversion to my other setup later. RTX3060 is 12, btw.

**CLAUDE**

12 GB confirmed — so bf16 up to about 3B without quantization, which is the range worth exploring anyway.

Parked for later. When you come back to it, the two gates are: does a CUDA torch wheel exist for Python 3.14 on Windows, and the module-path resolver. Everything else follows from those.

So: the layer sweep.

Set **Head** to mean of all 9, **Scale** to  $\times$  uniform, then step **Layer** through 1, 2, 3, 6, 12, 18, 24, 30. Read the "Column 0 as a multiple of uniform" line at the bottom each time — it gives you every row in one string, so you can paste it straight here without hunting through cells.

What we're testing: StreamingLLM reports that in Llama-2-7B, layers 0 and 1 show a *local* pattern — recent tokens getting the attention — with the sink emerging from roughly layer 3 onward. Your model is  $50\times$  smaller and a different training run, so this genuinely might not hold.

If layers 1–2 come back near  $1.0\times$  on column 0 while layer 3 onward climbs, you have a published prediction confirmed on your laptop. If the sink is there from layer 1, that's a real difference worth knowing about before you describe layer behaviour to anyone.

Either result closes the last open claim in the tool.

**YOU**

What does it look like to you?

**CLAUDE**

The prediction holds at the bottom, and the middle is more interesting than anyone told you to expect.

### **Layer 1 has no sink at all**

Column 0 in the layer-1 grid is almost entirely blank — below  $0.25\times$ , which means the final positions give position 0 *less* than uniform attention would. The bright cells sit on and beside the diagonal: 2.2, 2.7, 2.8, 3.9, 3.3, 3.2, 3.1, all hugging the right edge of each row.

That is precisely the local pattern StreamingLLM reports for Llama-2-7B's bottom layers, reproduced on a 135M model from a different training run. Layer 2 shows the sink appearing weakly in the early rows (1.6, 1.7, 1.6) while the diagonal still dominates. By layer 3 it's established across every row at roughly  $2-3\times$ .

Prediction confirmed. That's a published result you reproduced on your own laptop in ten minutes.

### **But it isn't a ramp**

Largest column-0 multiple anywhere in each grid:

**YOU**

Is there a chance we could add this graph to the UI, with detailed description and analysis

"To verify independently — Paste the eight column-0 summary lines; that replaces my image reading with exact figures. Then re-run layers 12 and 24 on the Japan prompt to see whether the pattern is prompt-independent." - I don't understand what I need to do here

**CLAUDE**

Two separate things. Taking the confusing one first.

### What "verify independently" meant

Nothing complicated — I was asking you to copy the sentence that already prints under each grid, the one starting "**Column 0 as a multiple of uniform: row 1 1.6x, row 2 2.0x...**", once per layer, so I'd have exact numbers instead of squinting at your screenshots.

But that's a bad ask. It's eight rounds of copy-paste to work around a limitation on my end.

Better idea, and it's the one you just had: **the browser already has all 30 layers.** `attnCache` holds every layer and head from the single Load click. The chart can be computed from your actual data, live, with zero server requests and zero copying. My screenshot-reading problem disappears entirely.

So — yes, let's build it.

**YOU**

What do you make of this?

**CLAUDE**

That inverts something I told you, and it's a much better result than the one I had.

**YOU**

**CLAUDE**

Wrong prediction. Badly wrong, and in an interesting direction.

At layer 5, bottom row, mean of 9 (baseline  $1/25 = 4\%$ ):

- like -> oats **0.37% = 0.09× uniform**
- like -> hay **0.49% = 0.12× uniform**

Both content nouns get roughly a *tenth* of what an even split would give. Not marginal. Actively suppressed.

### What the 10.9× actually is

The summary line names it: the #5 (43.7%, 10.9x), The #0 (22.6%, 5.7x), the #11 (18.2%, 4.6x) .

Position 5 is the . Position 11 is the . The strongest content-adjacent attention in the entire network is going to **two determiners**.

And it's not just the bottom row. Look at the grid — that's a vertical stripe. Every row from hay downward gives column 5 between 37% and 44%, and column 11 between 15% and 19%, row after row after row. That's the same stripe I spotted in the layer-2-through-6 screenshots and couldn't identify. Now it has a name, and the name is a definite article.

### So my two-regime story needs rewriting

I told you the model does relational work early and structural work late. That's not what this shows.

It does **one kind of structural work early** — concentrating on a determiner — and **a different kind late** — concentrating on position 0. Both are attractors that have nothing to do with what the sentence means. Semantic retrieval of oats or hay doesn't appear in the mean at either end.

**My chart label is the problem.** I called that orange line "content (best other)." It measures "the largest weight on something that isn't the sink or the diagonal" — which I assumed would be meaning, and at layer 5 is a determiner. The label is doing interpretive work the number doesn't support. It should read something like "best other (non-sink, non-self)" and let the reader hover to find out what it is.

I'd rather fix that before a coworker reads "content" off the legend and concludes the model is doing semantics at layer 5.

## The one comparison that's still open

Layer 30's `hay` at  $5.4\times$  was measured in **max across heads**. Layer 5's  $0.12\times$  is the **mean**. Those aren't comparable, and I shouldn't have set them against each other.

Switch to **max of all 9** at layer 5 and hover `oats` and `hay` again. If a single head is doing noun retrieval there, the mean would bury it exactly as it buried everything else all day.

Want me to relabel the chart line while you check?

---

**Solid** —  $0.37\%$  and  $0.49\%$  read from tooltips; baseline  $1/25 = 4\%$ , giving  $0.09\times$  and  $0.12\times$ ;  $43.7\% \times 25 = 10.9\times$ , confirming both the token count and the summary's arithmetic; positions 5 and 11 are both `the`; the vertical stripe at column 5 runs 37–44% across at least twelve consecutive rows.

**Uncertain / contested** — Why a determiner attracts this. I have no mechanism and won't invent one. Whether it holds on other prompts, or is specific to a sentence containing two instances of `the`. Whether any individual head at layer 5 does noun retrieval — untested, and the mean cannot answer it.

**To verify independently** — Layer 5, max of all 9, hover `oats` (#6) and `hay` (#12).